Bonus-Exercise: Parallelization of Modeling Ocean Circulation with Dask

Task B.1: Convert the NumPy-based serial implementation to Dask, enabling parallel computation.

In order to parallelize the code with dask we need to make some changes. First we exchange the numpy arrays with dask arrays. In order to calculate the laplacian operator we also need ghost cells, therefore we use map_overlap(). For the boundaries we use the "reflect" mode, since the ocean doesn't have any hard borders and the "constant"-mode would not be suitable. We also divide the grid in 4 chunks (no matter the grid size) first.

The changes can be seen below:

```python
import numpy as np
import matplotlib.pyplot as plt
import dask.array as da
import time

def laplacian(field):
    """Computes the discrete Laplacian of a 2D field using finite differences."""
    lap = (
        np.roll(field, shift=1, axis=0) +
        np.roll(field, shift=-1, axis=0) +
        np.roll(field, shift=1, axis=1) +
        np.roll(field, shift=-1, axis=1) -
        4 * field
    )
    return lap

def update_ocean(u, v, temperature, wind, alpha=0.1, beta=0.02):
    """Updates ocean velocity and temperature fields using a simplified flow model."""
    u_lap = u.map_overlap(laplacian, depth=1, boundary='reflect')
    v_lap = v.map_overlap(laplacian, depth=1, boundary='reflect')
    t_lap = temperature.map_overlap(laplacian, depth=1, boundary='reflect')

    u_new = da.map_blocks(lambda u, ul, w: u + alpha * ul + beta * w, u, u_lap, wind)
    v_new = da.map_blocks(lambda v, vl, w: v + alpha * vl + beta * w, v, v_lap, wind)
    temperature_new = da.map_blocks(lambda t, tl: t + 0.01 * tl, temperature, t_lap)

    return u_new, v_new, temperature_new

# Grid size
grid_sizes = [200, 2000, 5000]
TIME_STEPS = 100

for grid_size in grid_sizes:
    chunks = (grid_size/2,grid_size/2)
    # Init values
    temperature = da.random.uniform(5, 30, size=(grid_size, grid_size), chunks=chunks)
    u_velocity = da.random.uniform(-1, 1, size=(grid_size, grid_size), chunks=chunks)
    v_velocity = da.random.uniform(-1, 1, size=(grid_size, grid_size), chunks=chunks)
    wind = da.random.uniform(-0.5, 0.5, size=(grid_size, grid_size), chunks=chunks)

    # Run the simulation
    start = time.time()
    for t in range(TIME_STEPS):
        u_velocity, v_velocity, temperature = update_ocean(u_velocity, v_velocity, temperature, wind)
    u_final, v_final, t_final = da.compute(u_velocity, v_velocity, temperature)
    end = time.time()

    print(f"Simulation (grid size: {grid_size}) completed in {end - start} seconds.")
```

Also a timer was added in the original version and the plotting was removed. Both simulations run with 3 different grid sizes (200, 2000, 5000) in order to compare the execution times.

Let's run both and compare them:

```
In [1]: !python ocean_original.py
```

```
Simulation (grid size: 200) completed in 0.04039192199707031 seconds.
Simulation (grid size: 2000) completed in 4.406620740890503 seconds.
Simulation (grid size: 5000) completed in 25.57630705833435 seconds.
```

```
In [2]: !python ocean_dask.py
```

```
Simulation (grid size: 200) completed in 11.399661779403687 seconds.
Simulation (grid size: 2000) completed in 13.818193912506104 seconds.
Simulation (grid size: 5000) completed in 24.300597190856934 seconds.
```

As we can see, for the smaller grid the serial version is way faster but with increasing grid sizes the parallel approach with dask is catching up and with a 5000x5000 grid it is even a bit faster. This is due to the overhead dask setting up its parallelization, at some point this overhead is compensated by the speedup of calculating the grid in different portions. Let's run the dask version again but now lets change it, so the grid is devided in 16 chunks instead of 4.

```
In [4]: !python ocean_dask.py
```

```
Simulation (grid size: 200) completed in 45.40377902984619 seconds.
Simulation (grid size: 2000) completed in 51.9316349029541 seconds.
Simulation (grid size: 5000) completed in 61.85209107398987 seconds.
```

The execution times are increasing when deviding the grid in even smaller but more portions. Thats because now even more tasks have to be created and therefore also result in more overhead. This only makes sense if the grid size is very big.

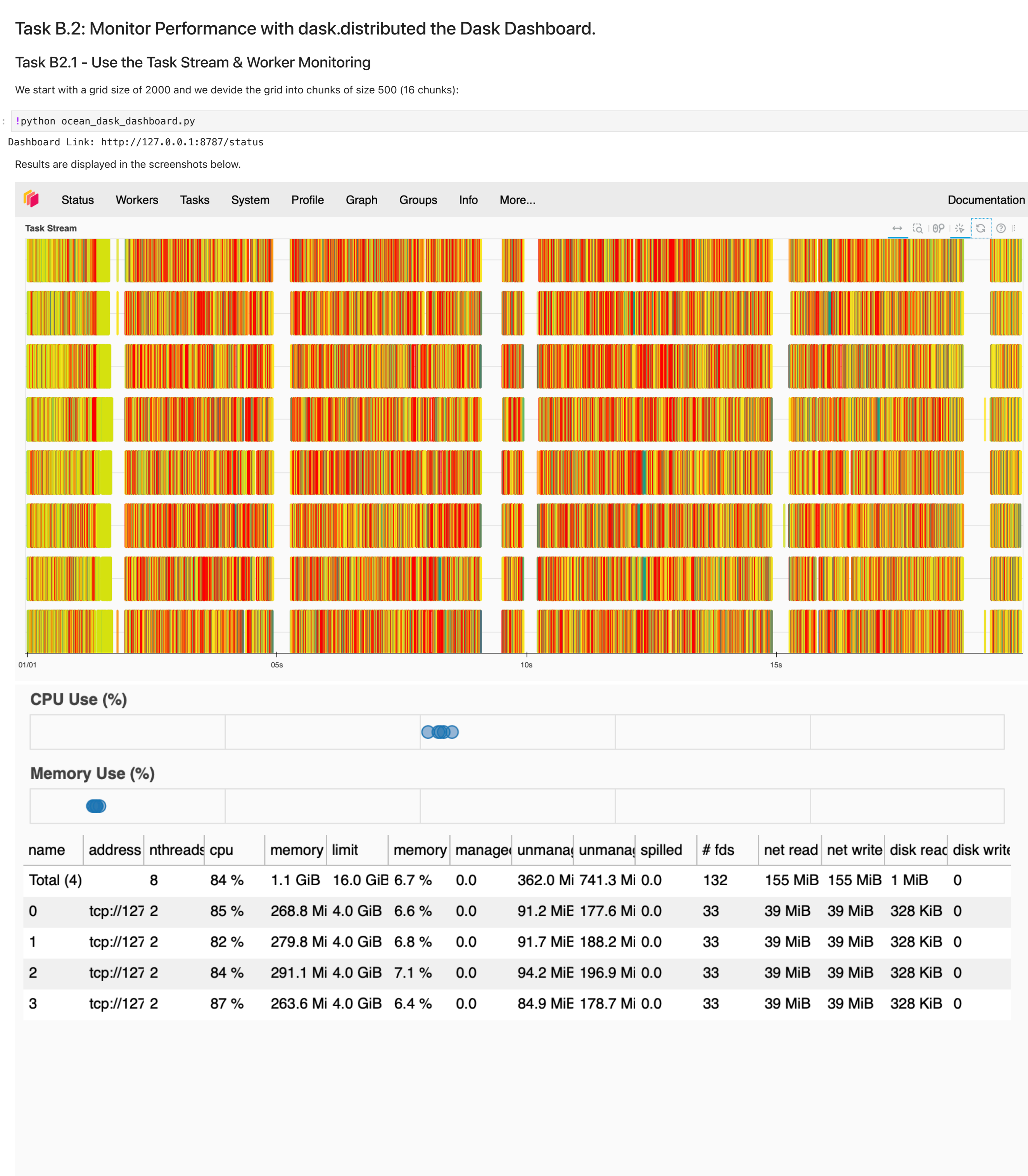
Task B.2: Monitor Performance with dask.distributed the Dask Dashboard.

Task B2.1 - Use the Task Stream & Worker Monitoring

We start with a grid size of 2000 and we devide the grid into chunks of size 500 (16 chunks):

```
In [14]: !python ocean_dask_dashboard.py
Dashboard Link: http://127.0.0.1:8787/status
```

Results are displayed in the screenshots below.



name	address	nthreads	cpu	memory	limit	memory	manage	unmana	unmana	spilled	# fds	net read	net write	disk reac	disk write
Total (4)		8	84 %	1.1 GiB	16.0 GiB	6.7 %	0.0	362.0 Mi	741.3 Mi	0.0	132	155 MiB	155 MiB	1 MiB	0
0	tcp://127	2	85 %	268.8 Mi	4.0 GiB	6.6 %	0.0	91.2 MiE	177.6 Mi	0.0	33	39 MiB	39 MiB	328 KiB	0
1	tcp://127	2	82 %	279.8 Mi	4.0 GiB	6.8 %	0.0	91.7 MiE	188.2 Mi	0.0	33	39 MiB	39 MiB	328 KiB	0
2	tcp://127	2	84 %	291.1 Mi	4.0 GiB	7.1 %	0.0	94.2 MiE	196.9 Mi	0.0	33	39 MiB	39 MiB	328 KiB	0
3	tcp://127	2	87 %	263.6 Mi	4.0 GiB	6.4 %	0.0	84.9 MiE	178.7 Mi	0.0	33	39 MiB	39 MiB	328 KiB	0

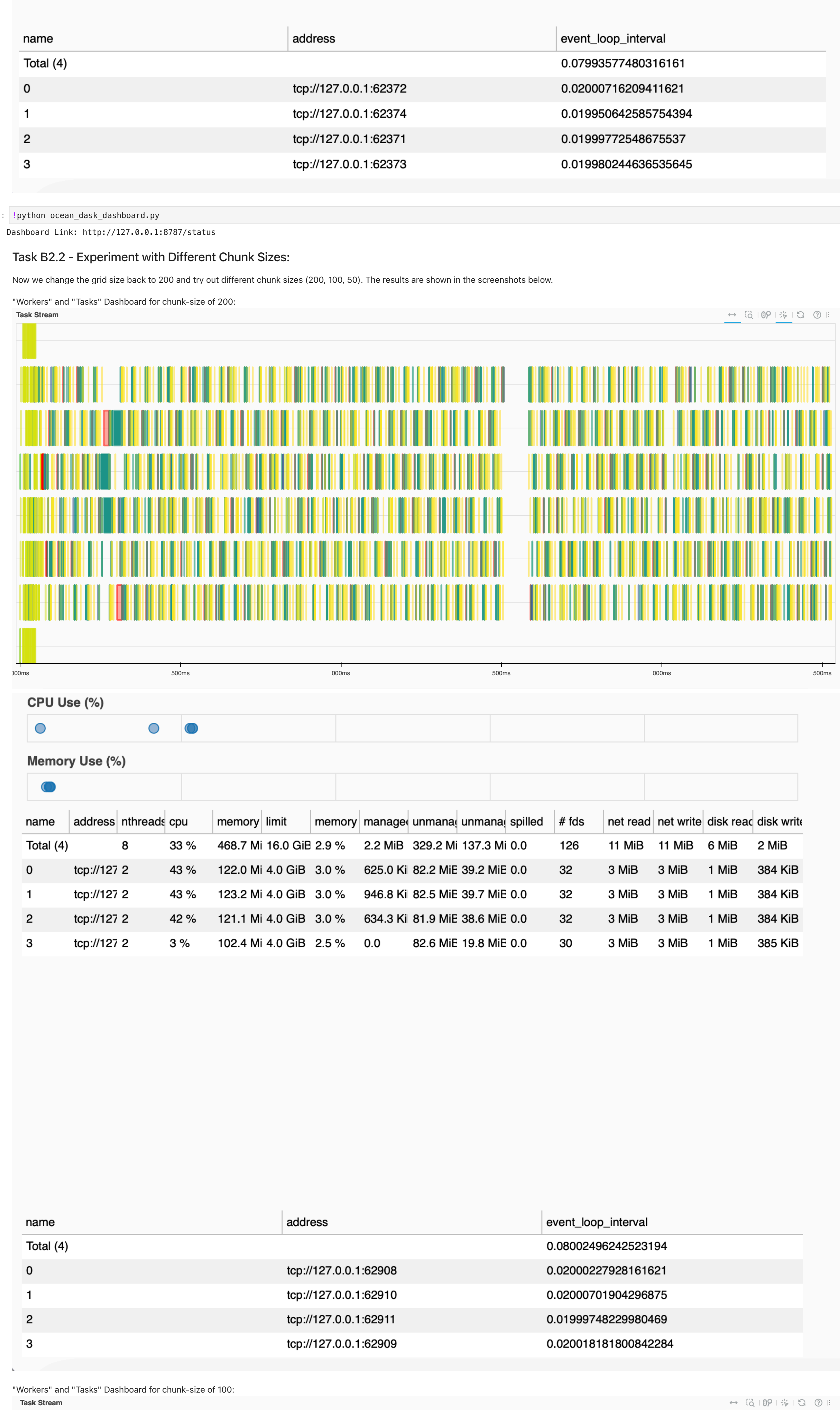
name	address	event_loop_interval
Total (4)		0.07993577480316161
0	tcp://127.0.0.1:62372	0.02000716209411621
1	tcp://127.0.0.1:62374	0.019950642585754394
2	tcp://127.0.0.1:62371	0.01999772548675537
3	tcp://127.0.0.1:62373	0.019980244636535645

```
In [22]: !python ocean_dask_dashboard.py
Dashboard Link: http://127.0.0.1:8787/status
```

Task B2.2 - Experiment with Different Chunk Sizes:

Now we change the grid size back to 200 and try out different chunk sizes (200, 100, 50). The results are shown in the screenshots below.

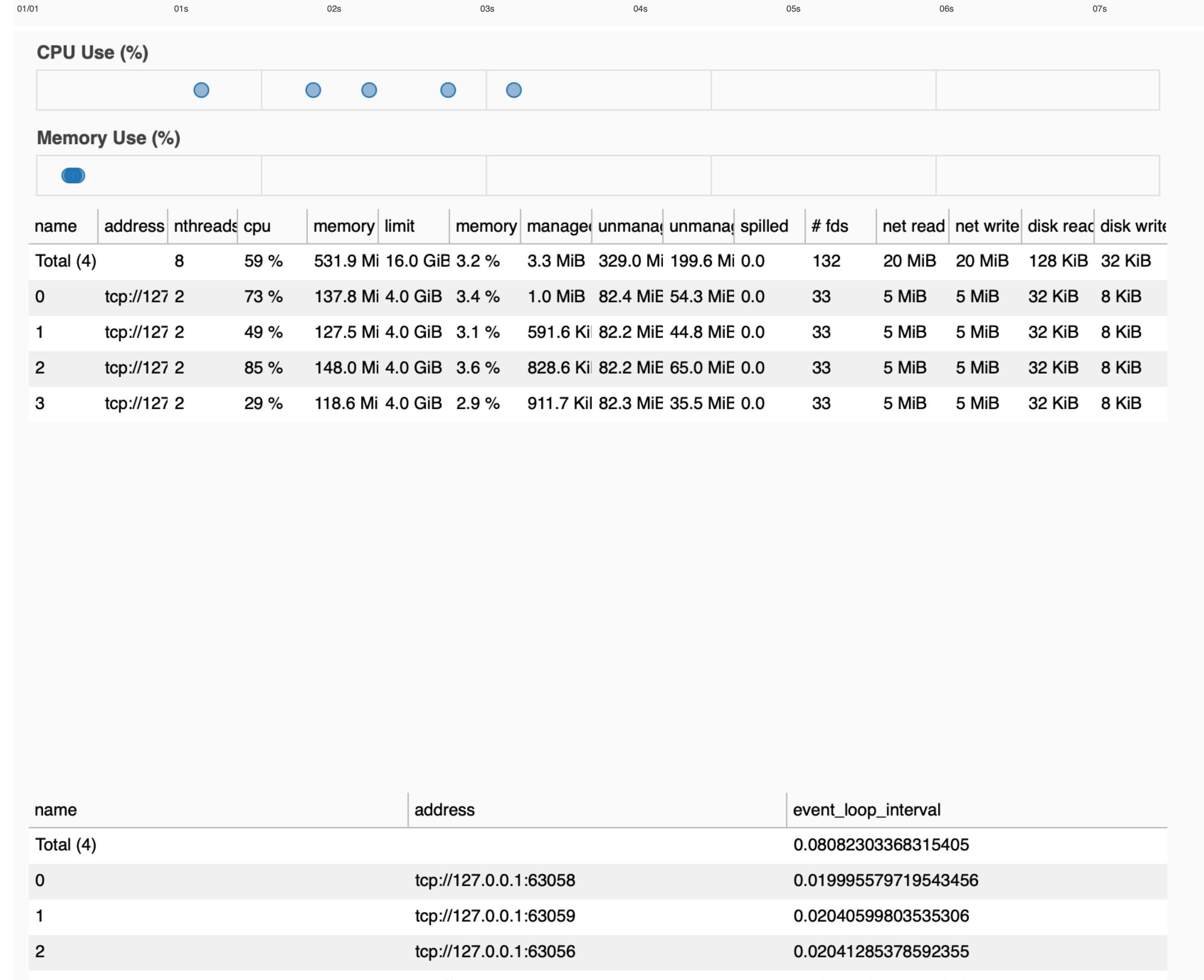
"Workers" and "Tasks" Dashboard for chunk-size of 200:



name	address	nthreads	cpu	memory	limit	memory	manage	unmana	unmana	spilled	# fds	net read	net write	disk reac	disk write
Total (4)		8	33 %	468.7 Mi	16.0 GiB	2.9 %	2.2 MiB	329.2 Mi	137.3 Mi	0.0	126	11 MiB	11 MiB	6 MiB	2 MiB
0	tcp://127	2	43 %	122.0 Mi	4.0 GiB	3.0 %	625.0 Ki	82.2 MiE	39.2 MiE	0.0	32	3 MiB	3 MiB	1 MiB	384 KiB
1	tcp://127	2	43 %	123.2 Mi	4.0 GiB	3.0 %	946.8 Ki	82.5 MiE	39.7 MiE	0.0	32	3 MiB	3 MiB	1 MiB	384 KiB
2	tcp://127	2	42 %	121.1 Mi	4.0 GiB	3.0 %	634.3 Ki	81.9 MiE	38.6 MiE	0.0	32	3 MiB	3 MiB	1 MiB	384 KiB
3	tcp://127	2	3 %	102.4 Mi	4.0 GiB	2.5 %	0.0	82.6 MiE	19.8 MiE	0.0	30	3 MiB	3 MiB	1 MiB	385 KiB

name	address	event_loop_interval
Total (4)		0.08002496242523194
0	tcp://127.0.0.1:62908	0.02000227928161621
1	tcp://127.0.0.1:62910	0.02000701904296875
2	tcp://127.0.0.1:62911	0.01999748229980469
3	tcp://127.0.0.1:62909	0.02001818180842284

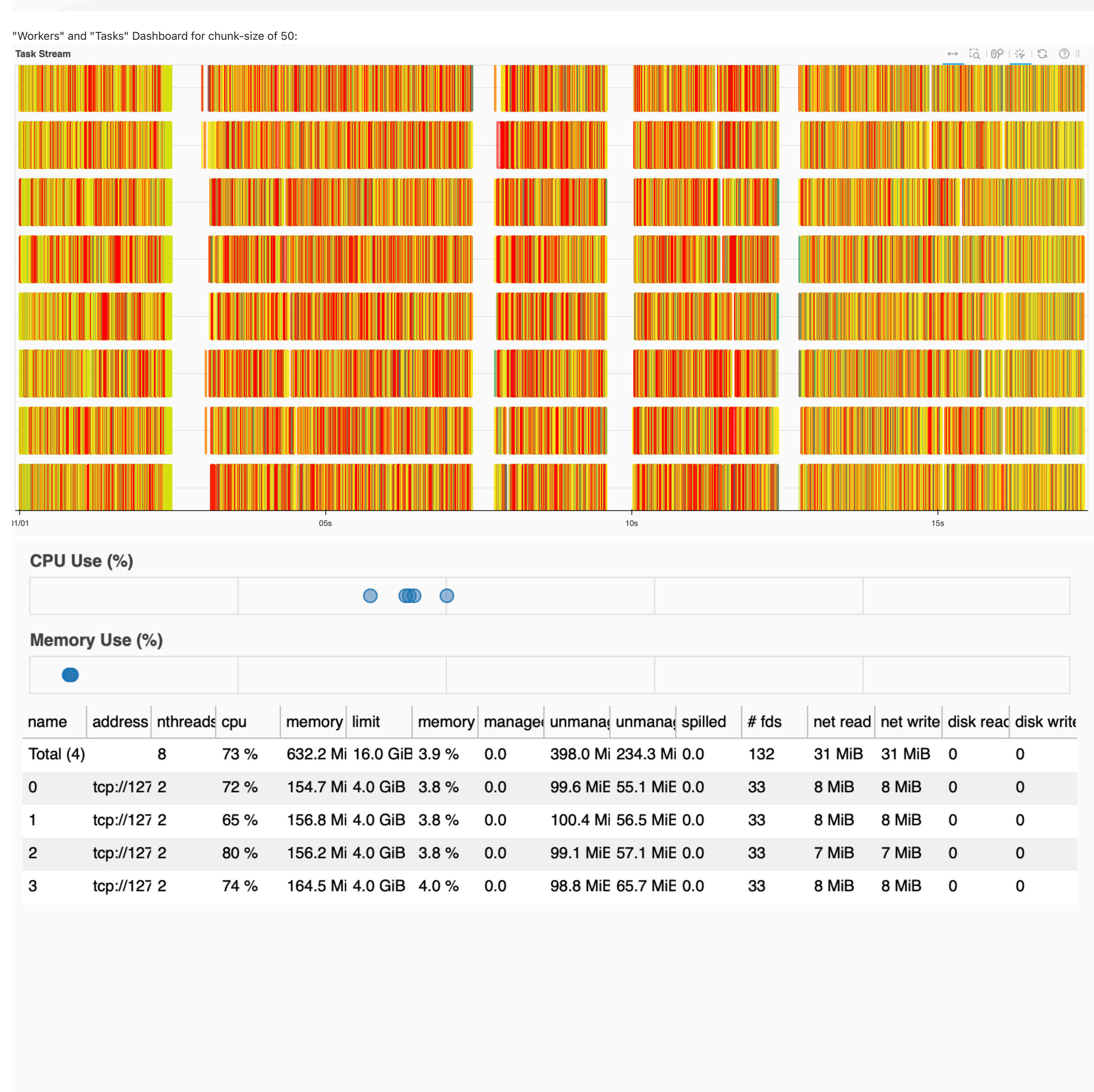
"Workers" and "Tasks" Dashboard for chunk-size of 100:



name	address	nthreads	cpu	memory	limit	memory	manage	unmana	unmana	spilled	# fds	net read	net write	disk reac	disk write
Total (4)		8	59 %	531.9 Mi	16.0 GiB	3.2 %	3.3 MiB	329.0 Mi	199.6 Mi	0.0	132	20 MiB	20 MiB	128 KiB	32 KiB
0	tcp://127	2	73 %	137.8 Mi	4.0 GiB	3.4 %	1.0 MiB	82.4 MiE	54.3 MiE	0.0	33	5 MiB	5 MiB	32 KiB	8 KiB
1	tcp://127	2	49 %	127.5 Mi	4.0 GiB	3.1 %	591.6 Ki	82.2 MiE	44.8 MiE	0.0	33	5 MiB	5 MiB	32 KiB	8 KiB
2	tcp://127	2	85 %	148.0 Mi	4.0 GiB	3.6 %	828.6 Ki	82.2 MiE	65.0 MiE	0.0	33	5 MiB	5 MiB	32 KiB	8 KiB
3	tcp://127	2	29 %	118.6 Mi	4.0 GiB	2.9 %	911.7 Kil	82.3 MiE	35.5 MiE	0.0	33	5 MiB	5 MiB	32 KiB	8 KiB

name	address	event_loop_interval
Total (4)		0.08082303368315405
0	tcp://127.0.0.1:63058	0.019995579719543456
1	tcp://127.0.0.1:63059	0.02040599803535306
2	tcp://127.0.0.1:63056	0.02041285378592355
3	tcp://127.0.0.1:63057	0.020008602142333985

"Workers" and "Tasks" Dashboard for chunk-size of 50:



name	address	nthreads	cpu	memory	limit	memory	manage	unmana	unmana	spilled	# fds	net read	net write	disk reac	disk write
Total (4)		8	73 %	632.2 Mi	16.0 GiB	3.9 %	0.0	398.0 Mi	234.3 Mi	0.0	132	31 MiB	31 MiB	0	0
0	tcp://127	2	72 %	154.7 Mi	4.0 GiB	3.8 %	0.0	99.6 MiE	55.1 MiE	0.0	33	8 MiB	8 MiB	0	0
1	tcp://127	2	65 %	156.8 Mi	4.0 GiB	3.8 %	0.0	100.4 Mi	56.5 MiE	0.0	33	8 MiB	8 MiB	0	0
2	tcp://127	2	80 %	156.2 Mi	4.0 GiB	3.8 %	0.0	99.1 MiE	57.1 MiE	0.0	33	7 MiB	7 MiB	0	0
3	tcp://127	2	74 %	164.5 Mi	4.0 GiB	4.0 %	0.0	98.8 MiE	65.7 MiE	0.0	33	8 MiB	8 MiB	0	0

name	address	event_loop_interval
Total (4)		0.0810053930679957
0	tcp://127.0.0.1:62728	0.019819040298461914
1	tcp://127.0.0.1:62729	0.019915761947631835
2	tcp://127.0.0.1:62727	0.021323586503664654
3	tcp://127.0.0.1:62730	0.019947004318237305

The smaller the chunk size, the higher is the CPU usage. The memory usage seems to stay almost the same, no matter the chunk size. But also the overhead increases with a smaller chunk size.

The simulation time overall was not very long and also the CPU and Memory Usage jumped during the simulation, so it is hard to compare them against each other.

Task B2.3 - Answer the following questions:

How well-balanced were the worker loads?

Since the CPU usage jumped quite a lot during the execution it is hard to tell how well they were balanced, especially for the small grid of 200. But for the simulation with a grid size of 2000 the cpu usage for all workers were well balanced, as we can see in the first screenshot.

Did any worker run out of memory?

The memory usage seemed to be quite low, but the task visualization shows some red bars, so there might have been some out of memory problems.

Was there idle time or task queuing?

In all of the simulations there was quite some idle time, this also explains why the execution time of the serial code was faster than the parallel approach (mainly because of the overhead but also because the dask execution and distribution to the workers was not optimal).

Task B2.4 - Save the flow information on the grid in VTK files and use Paraview for the visualization.

In order to create the vtk files we modify code by adding the following function and execute it every 10 timesteps (we use the serial version of the code again because it runs the fastest for this small grid size):

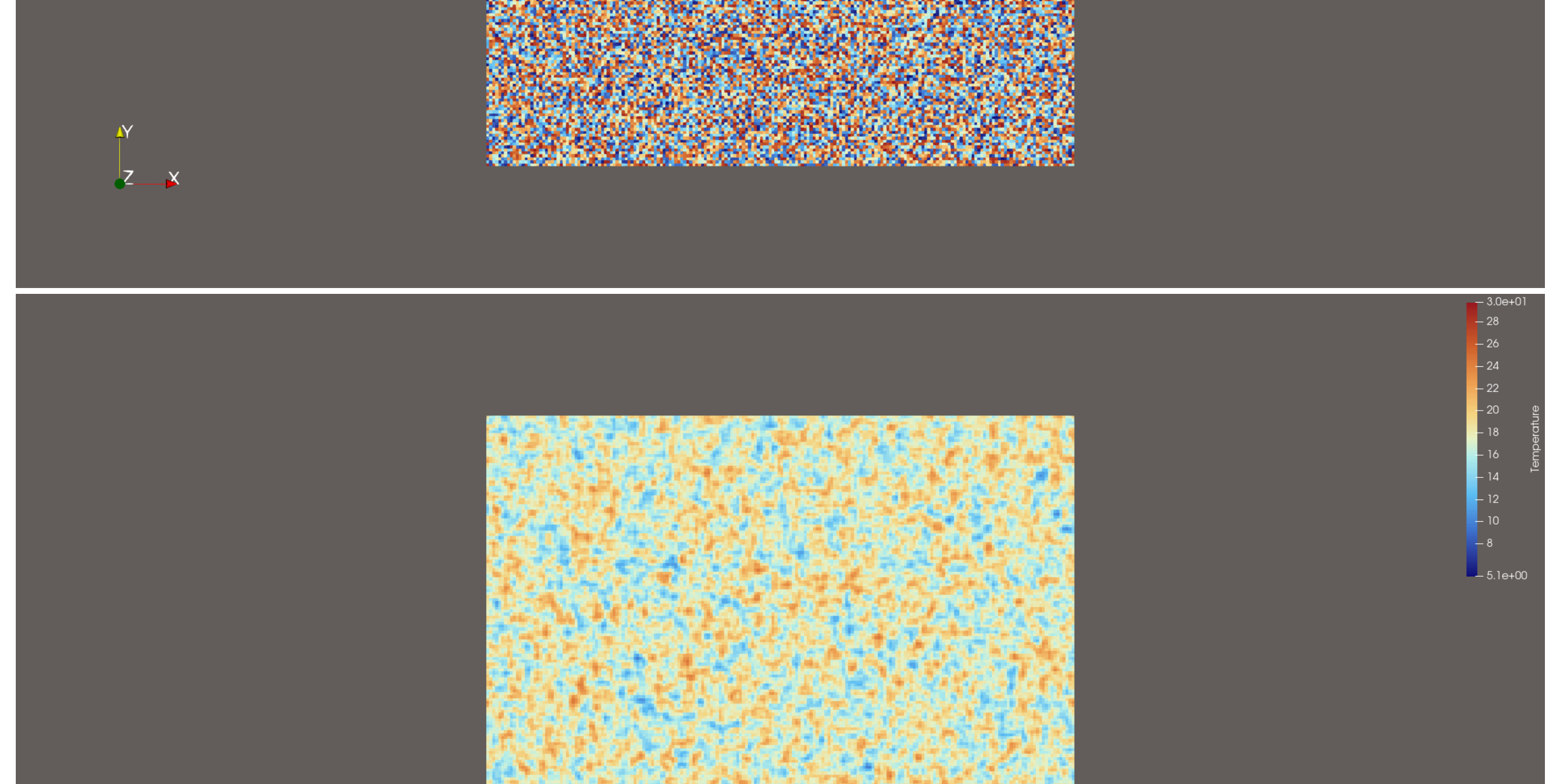
```python
def save_ocean_vtk(filename, u, v, temp):
    nx, ny = temp.shape

    u_flat = u.T.flatten()
    v_flat = v.T.flatten()
    temp_flat = temp.T.flatten()

    vtk_data = pyvtk.VtkData(
        pyvtk.StructuredPoints([nx, ny, 1]),
        pyvtk.PointData(
            pyvtk.Scalars(temp_flat, name="Temperature"),
            pyvtk.Vectors(np.column_stack((u_flat, v_flat, np.zeros_like(u_flat))), name="Velocity")
        )
    )
    vtk_data.tofile(filename)

# Run the simulation
for t in range(TIME_STEPS):
    u_velocity, v_velocity, temperature = update_ocean(u_velocity, v_velocity, temperature, wind)
    if t % 10 == 0 or t == TIME_STEPS - 1:
        fname = f"vtk_results/ocean_sim_{t:04d}.vtk"
        save_ocean_vtk(fname, u_velocity, v_velocity, temperature)
        print(f"Time Step {t}: Saved {fname}")
```

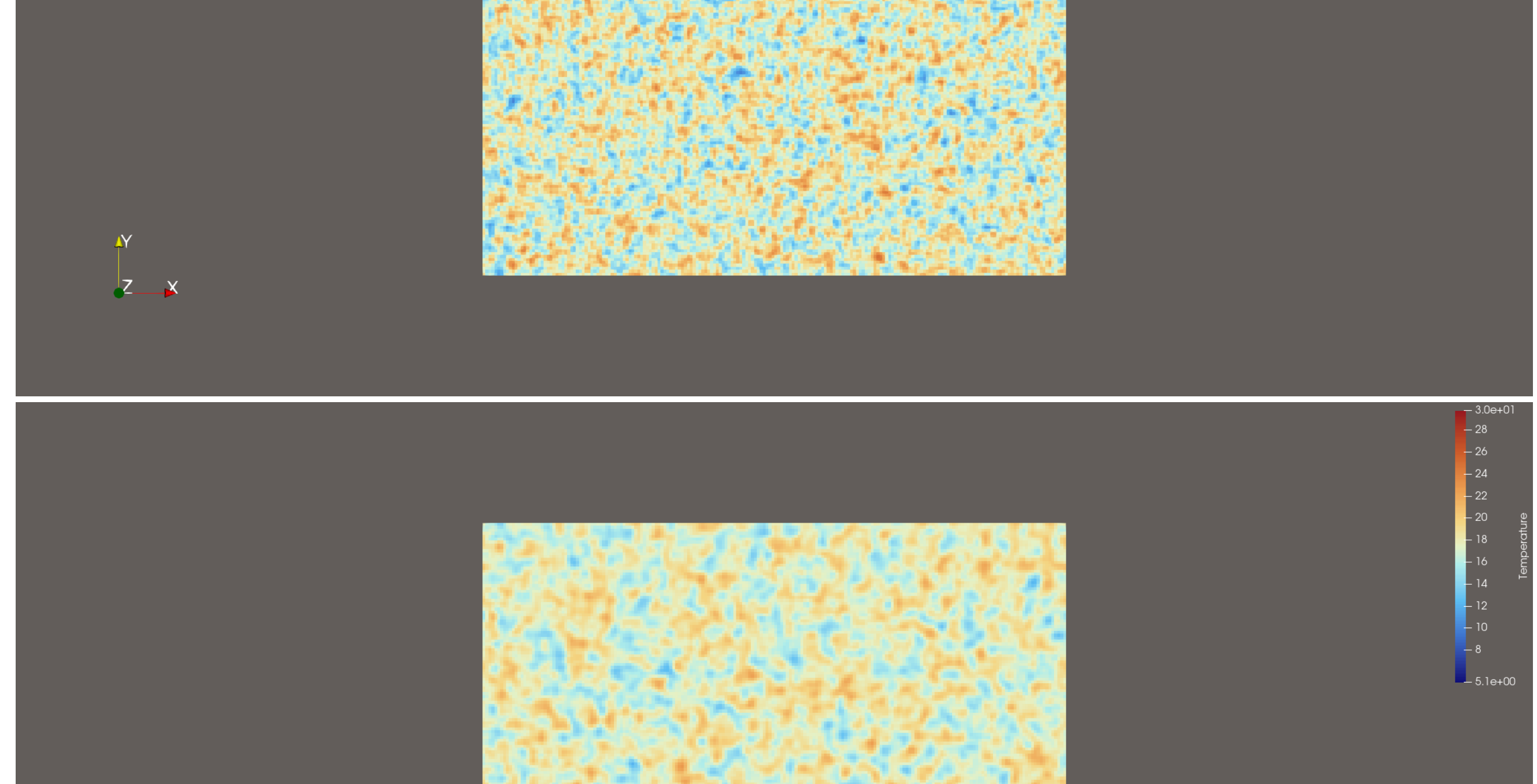
Let's run the code to save the files.

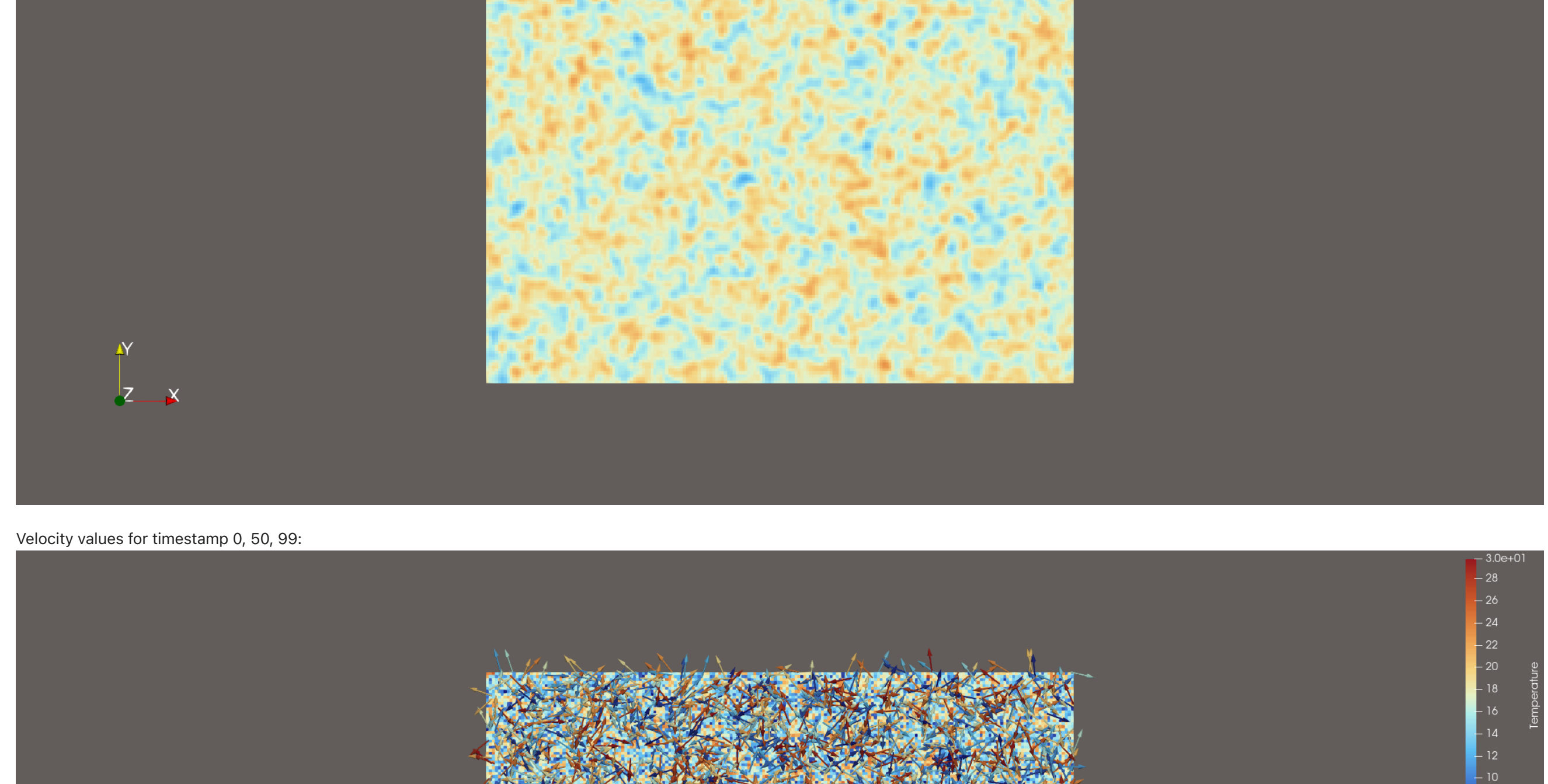
```
In [24]: !python ocean_vtk.py
```

```
Time Step 0: Saved vtk_results/ocean_sim_0000.vtk
Time Step 10: Saved vtk_results/ocean_sim_0010.vtk
Time Step 20: Saved vtk_results/ocean_sim_0020.vtk
Time Step 30: Saved vtk_results/ocean_sim_0030.vtk
Time Step 40: Saved vtk_results/ocean_sim_0040.vtk
Time Step 50: Saved vtk_results/ocean_sim_0050.vtk
Time Step 60: Saved vtk_results/ocean_sim_0060.vtk
Time Step 70: Saved vtk_results/ocean_sim_0070.vtk
Time Step 80: Saved vtk_results/ocean_sim_0080.vtk
Time Step 90: Saved vtk_results/ocean_sim_0090.vtk
Time Step 99: Saved vtk_results/ocean_sim_0099.vtk
```

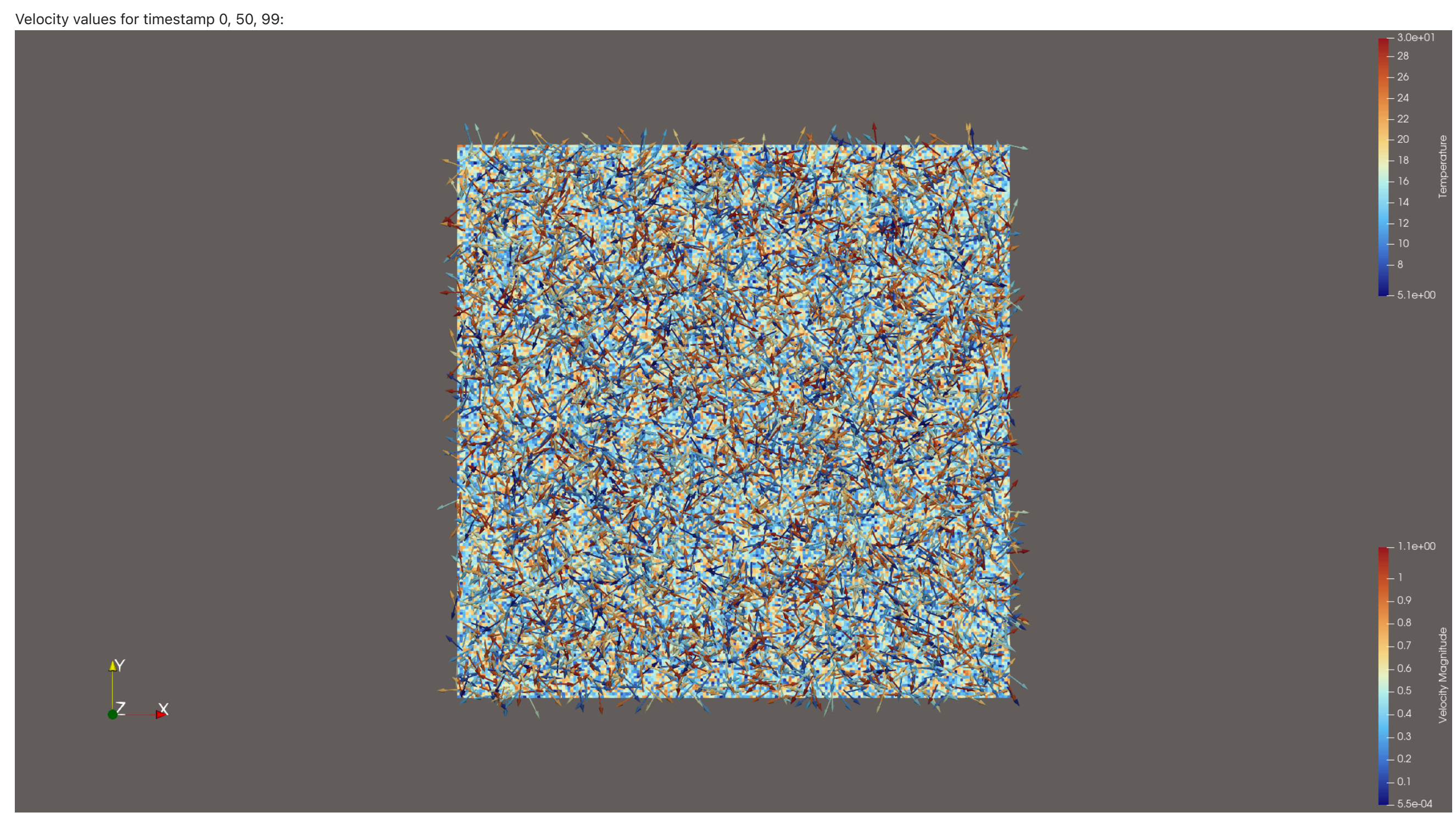
The results (visualized with ParaView) can be seen in the following screenshots: Temperature values for timestamp 0, 50, 99:

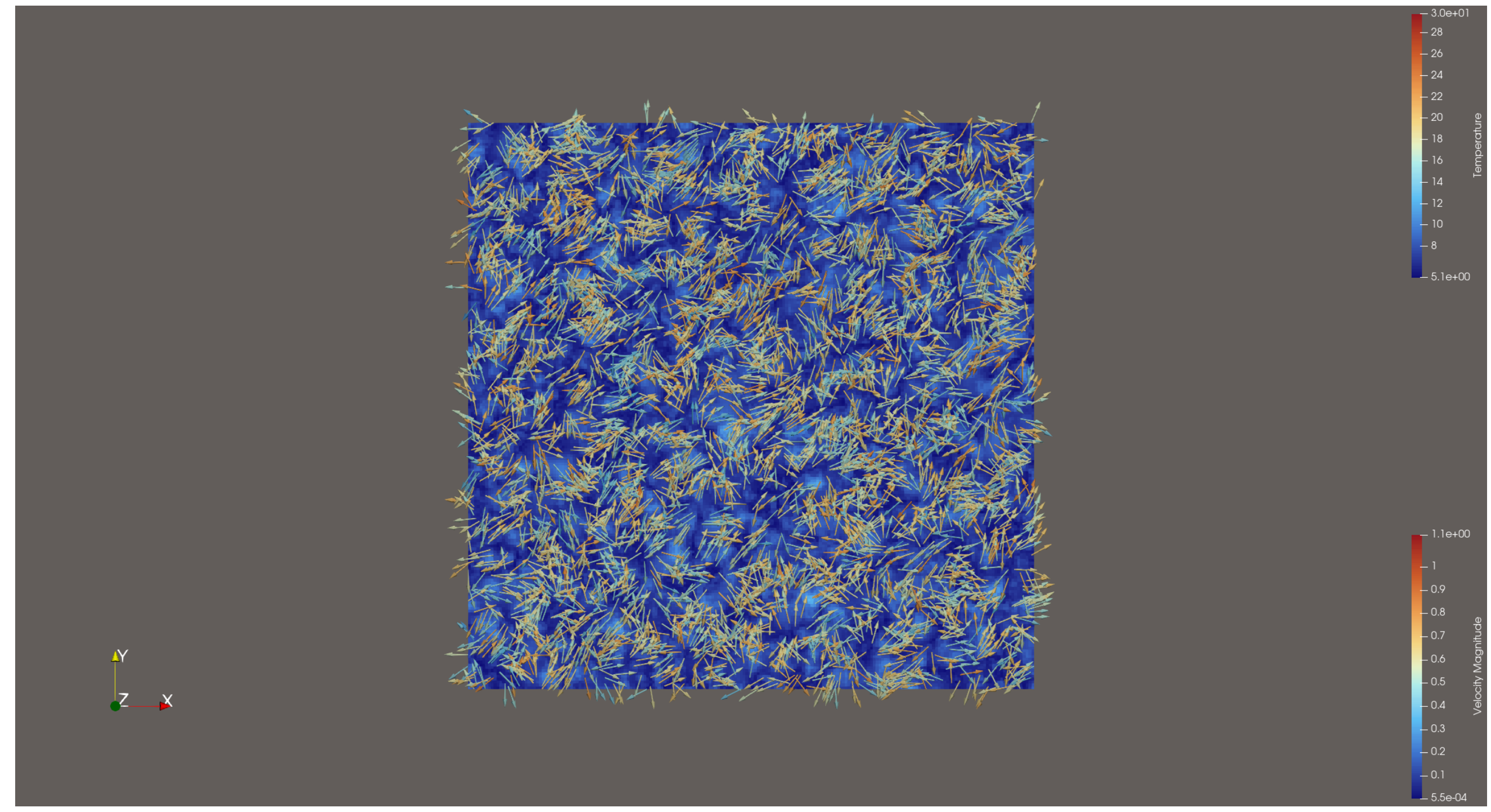


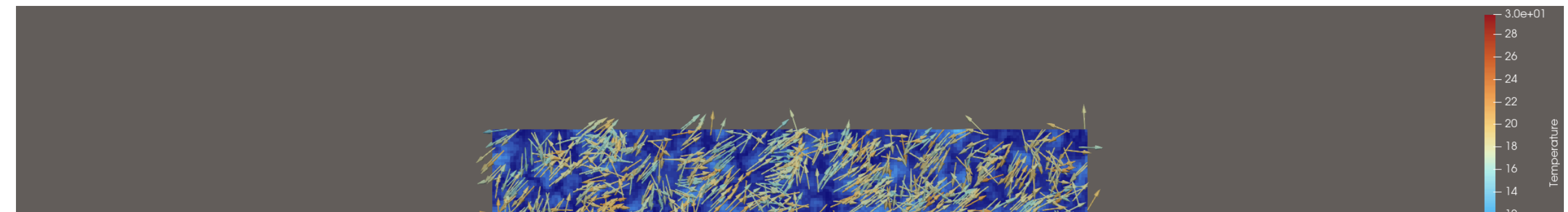


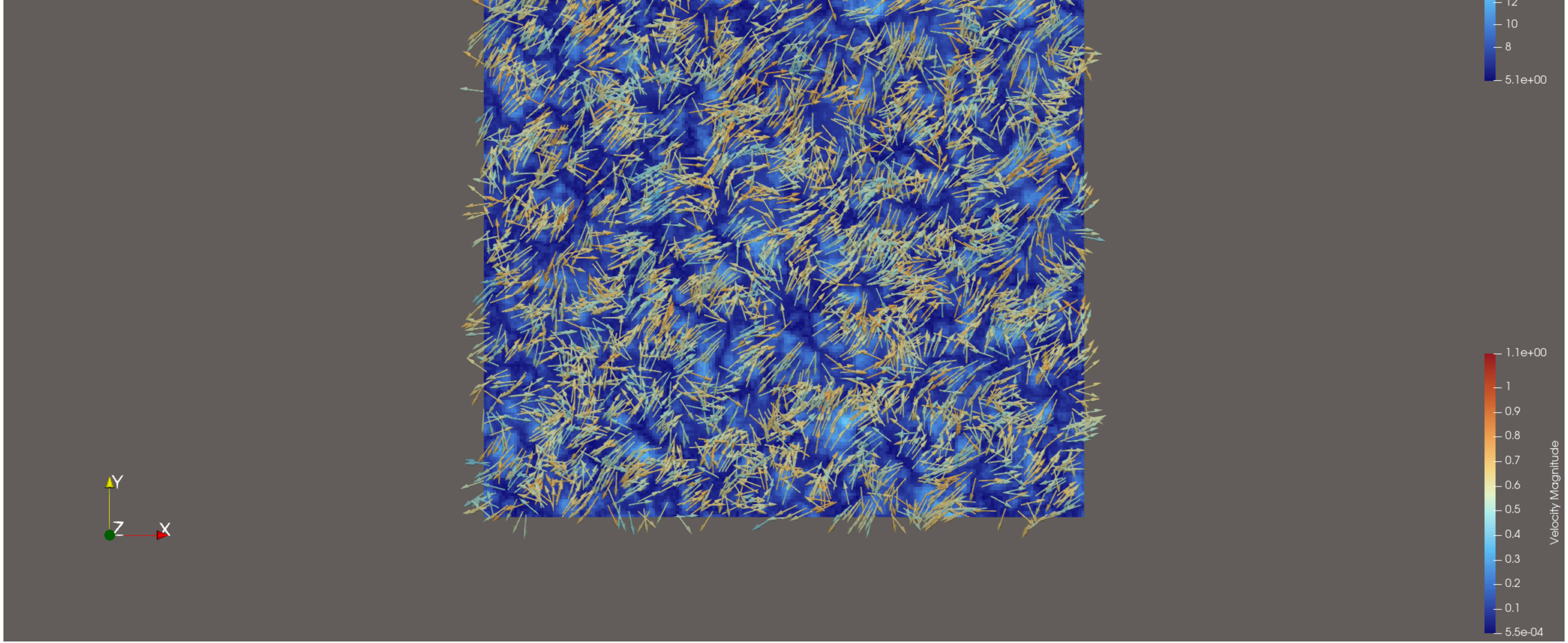


Velocity values for timestamp 0, 50, 99:








12
10
8
5.1e+00
Y
Z
X
1.1e+00
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
5.5e-04
Velocity Magnitude